Documentación Técnica y de API

Migración del Dominio de Usuarios de Monolito a Microservicio en ShopSmart

Asignatura: Desarrollo Fullstack III
Fecha: 17 de marzo de 2026

Resumen Ejecutivo

Este documento presenta la documentación técnica completa del microservicio de Usuarios desarrollado con **Spring Boot 3.2** para la plataforma ShopSmart. Se incluye la especificación de requerimientos funcionales y no funcionales, documentación de la API REST, diagramas UML de diseño lógico, diagrama BPMN del proceso operativo principal, y la justificación arquitectónica de la migración desde un sistema monolítico.

Framework:	Spring Boot 3.2 + Java 17
Base de datos:	PostgreSQL 16
Autenticación:	JWT (access + refresh)
Infraestructura:	Docker / Docker Compose
Documentación:	OpenAPI 3 / Swagger UI

0 Índice

1. Introducción **3**
1.1. Alcance del documento . . . 3

2. Planteamiento del Problema **3**
2.1. Limitaciones del monolito en el dominio de Usuarios . . . 3
2.2. Impacto en la experiencia del cliente . . . 4

3. Objetivos **4**
3.1. Objetivo general . . . 4
3.2. Objetivos específicos . . . 4

4. Metodología **5**

5. Requerimientos Funcionales **5**

6. Requerimientos No Funcionales **6**

7. Diseño Lógico: Diagrama UML de Clases **7**
7.1. Descripción de las entidades . . . 8

8. Diagrama BPMN: Ciclo de Vida de Autenticación **8**
8.1. Descripción del flujo . . . 9

9. Documentación de la API REST **9**
9.1. Información general . . . 10
9.2. Módulo de Autenticación (`/api/v1/auth`) . . . 10
9.2.1. POST /api/v1/auth/registro — Registrar usuario . . . 10
9.2.2. POST /api/v1/auth/login — Iniciar sesión . . . 11
9.3. Módulo de Perfil (`/api/v1/usuarios`) . . . 11
9.4. Módulo Administrativo (`/api/v1/admin`) . . . 11
9.4.1. Paginación . . . 12
9.5. Estructura de errores . . . 12

10. Pruebas de Endpoints **13**
10.1. Alcance de las pruebas . . . 13
10.2. Casos de prueba por endpoint . . . 13
10.3. Ejecución de pruebas . . . 13

11. Arquitectura del Microservicio **14**
11.1. Estructura de capas . . . 14
11.2. Flujo de seguridad JWT . . . 14

12. Infraestructura y Despliegue **14**

12.1. Contenerización con Docker 15
12.2. Docker Compose 15
12.3. Variables de entorno 16

13. Argumentación Arquitectónica: Monolito a Microservicio 16
13.1. Justificación de Spring Boot sobre Django 17

14. Discusión de Impacto 17
14.1. Beneficios esperados 17
14.2. Riesgos y mitigaciones 17

15. Criterios de Aceptación 18

16. Conclusiones 18

1 Introducción

El crecimiento acelerado de ShopSmart como plataforma de comercio electrónico ha expuesto las limitaciones estructurales de su arquitectura monolítica. Conforme la carga transaccional aumentó, emergieron problemas de escalabilidad selectiva, acoplamiento de dominios y riesgo operativo en despliegues. La literatura especializada señala que en escenarios de alta evolución funcional el monolito se convierte en un limitante por acoplamiento y baja independencia de equipos [Newman(2021)] [Fowler(2015)].

En este contexto, el presente documento describe la migración incremental del dominio de **Usuarios** hacia un microservicio independiente, implementado con **Spring Boot 3.2** y **PostgreSQL**. La elección de Spring Boot frente a alternativas como Django se fundamenta en su ecosistema maduro para microservicios, su capacidad de manejo de concurrencia sin restricciones de GIL, y su integración nativa con herramientas de resiliencia como Resilience4j y Spring Cloud Gateway.

1.1 Alcance del documento

Este informe cubre:

- Especificación completa de requerimientos funcionales (RF) y no funcionales (RNF).
- Documentación de la API REST con todos sus *endpoints*, parámetros y códigos de respuesta.
- Diagrama UML de clases del diseño lógico del microservicio.
- Diagrama BPMN del flujo operativo principal (ciclo de vida de autenticación).
- Justificación arquitectónica de la migración monolito → microservicio.
- Discusión de impacto y riesgos de la arquitectura distribuida.

2 Planteamiento del Problema

2.1 Limitaciones del monolito en el dominio de Usuarios

Aunque las incidencias más visibles de ShopSmart aparecieron en los módulos de inventario y pedidos, el dominio de Usuarios también presentaba restricciones estructurales críticas:

- **Dependencia del ciclo de despliegue global:** cualquier cambio en el módulo de usuarios requería redesplegar todo el sistema, aumentando el riesgo y reduciendo la frecuencia de entregas.

- **Escalado no selectivo:** para crecer la capacidad de autenticación era necesario escalar todo el monolito, lo que resultaba costoso e ineficiente.
- **Acoplamiento de seguridad:** la mezcla de responsabilidades impedía endurecer el control de acceso sin riesgo de efectos colaterales en otros módulos.
- **Dificultad de evolución funcional:** agregar capacidades de fidelización, preferencias y soporte personalizado requería modificar una base de código compartida, ralentizando la entrega de valor.

2.2 Impacto en la experiencia del cliente

Estas limitaciones se traducían en consecuencias concretas para el negocio: períodos de mantenimiento que afectaban el acceso de usuarios, incapacidad de personalizar la experiencia de compra, y riesgo de exposición de datos sensibles por falta de aislamiento de la capa de identidad [Richardson(2018)].

3 Objetivos

3.1 Objetivo general

Argumentar técnica y funcionalmente la migración del dominio de Usuarios desde una arquitectura monolítica hacia un microservicio independiente implementado con Spring Boot 3.2, documentando su API, diseño lógico y flujos operativos.

3.2 Objetivos específicos

1. Especificar los requerimientos funcionales y no funcionales del microservicio de Usuarios alineados al contexto ShopSmart.
2. Documentar la API REST del microservicio con todos sus contratos de interfaz.
3. Modelar la solución mediante diagramas UML (clases) y BPMN (proceso de autenticación).
4. Fundamentar la decisión arquitectónica con criterios de escalabilidad, disponibilidad y seguridad.
5. Identificar riesgos y estrategias de mitigación propias de la arquitectura distribuida.

4 Metodología

Se aplicó una metodología de análisis aplicado con enfoque de ingeniería de software, estructurada en cinco fases:

1. **Diagnóstico arquitectónico:** análisis del contexto ShopSmart, identificación de límites de contexto (*bounded contexts*) y dependencias del dominio de Usuarios en el monolito.
2. **Diseño del microservicio:** definición de la estructura de capas (controladores, servicios, repositorios, modelos, DTOs), contratos de API y modelo de datos relacional.
3. **Implementación:** desarrollo con Spring Boot 3.2, Spring Security, JWT, Spring Data JPA y PostgreSQL. Contenerización con Docker.
4. **Documentación API:** generación automática con SpringDoc OpenAPI 3 y Swagger UI, complementada con este documento técnico formal.
5. **Modelado:** elaboración de diagramas UML de clases y BPMN del proceso de autenticación para trazabilidad de la solución.

5 Requerimientos Funcionales

Los requerimientos funcionales definen el comportamiento esperado del sistema desde la perspectiva de sus actores: usuarios estándar (clientes), administradores y servicios externos.

Cuadro 1: Requerimientos Funcionales del Microservicio de Usuarios

ID	Nombre	Descripción	Prioridad
RF-01	Registro de usuario	El sistema debe permitir crear cuentas con `nombre`, `apellido`, `email` y `password`. La contraseña se almacena hasheada con BCrypt. El email debe ser único en el sistema.	**Alta**
RF-02	Inicio de sesión	El sistema debe autenticar usuarios con credenciales válidas y emitir tokens JWT (`access` 24h y `refresh` 7 días).	**Alta**
RF-03	Cierre de sesión	El sistema debe soportar invalidación del token para cierre de sesión seguro y compatibilidad con arquitectura distribuida.	**Alta**

(continuación)

ID	Nombre	Descripción	Prioridad
RF-04	Consulta de perfil propio	El usuario autenticado puede consultar su información en el *endpoint* `GET /api/v1/usuarios/me`.	**Alta**
RF-05	Edición de perfil propio	El usuario autenticado puede actualizar `nombre`, `apellido` y `telefono` mediante `PUT /api/v1/usuarios/me`.	**Alta**
RF-06	Gestión administrativa	Un administrador puede listar, buscar y desactivar usuarios mediante endpoints bajo `/api/v1/admin/usuarios`.	**Alta**
RF-07	Protección de roles sensibles	El sistema no debe permitir que un usuario estándar se autoasigne privilegios administrativos (`ADMIN`, `SOPORTE`).	**Alta**
RF-08	Gestión de direcciones	El usuario autenticado puede agregar, listar y eliminar direcciones de envío asociadas a su perfil.	**Alta**
RF-09	Preferencias de personalización	El sistema almacena preferencias de usuario: categorías favoritas, consentimiento de marketing, idioma y moneda preferida.	**Media**
RF-10	Notas internas de soporte	El sistema permite almacenar `support_notes` para gestión interna de atención al cliente (visible solo para `ADMIN` y `SOPORTE`).	**Media**

6 Requerimientos No Funcionales

Cuadro 2: Requerimientos No Funcionales

ID	Categoría	Requerimiento	Métrica / Criterio
RNF-01	Seguridad	Autenticación JWT para todos los endpoints protegidos.	100 % de endpoints privados requieren token válido.
RNF-02	Seguridad	Control de acceso por rol (ADMIN vs CLIENTE).	Acciones administrativas devuelven HTTP 403 si el rol es insuficiente.

(continuación)

ID	Categoría	Requerimiento	Métrica
RNF-03	Seguridad	Contraseñas con hash BCrypt (salt rounds=10).	Nunca persistir contraseñas en texto plano.
RNF-04	Disponibilidad	Desacoplado de otros dominios críticos.	Caída de inventario/-pedidos no debe impedir acceso de usuarios.
RNF-05	Rendimiento	Respuesta rápida en autenticación y perfil.	p95 $<$ 300 ms en `/login` y `/me` en entorno de producción.
RNF-06	Escalabilidad	Escalado horizontal independiente.	Despliegue múltiples instancias sin escalar el sistema completo.
RNF-07	Mantenibilidad	Código modular por capas (DDD-lite).	Estructura por dominio: controller, service, repository, model, dto.
RNF-08	Observabilidad	Trazabilidad de errores y eventos.	Logs estructurados; Spring Actuator habilitado en `/actuator`.
RNF-09	Compatibilidad API	Contratos API estables y versionables.	Cambios *breaking* deben versionarse bajo `/api/v2/`.
RNF-10	Integridad	Consistencia tras migraciones de base de datos.	Migraciones aplicadas sin pérdida de datos; DDL controlado por JPA.
RNF-11	Documentación	API documentada con OpenAPI 3.	Swagger UI habilitado y actualizado en `/swagger-ui.html`.
RNF-12	Cumplimiento	Gestión de datos personales según normativa.	Evidencia de consentimiento (`marketing_opt_in`) y control de acceso.

7 Diseño Lógico: Diagrama UML de Clases

El diagrama de clases a continuación refleja el modelo de dominio implementado en el microservicio. Se distinguen tres entidades principales del dominio (`Usuario`, `DireccionEnvio`, `PreferenciaUsuario`) y las clases de transferencia de datos (DTOs)

utilizadas en la capa de presentación.

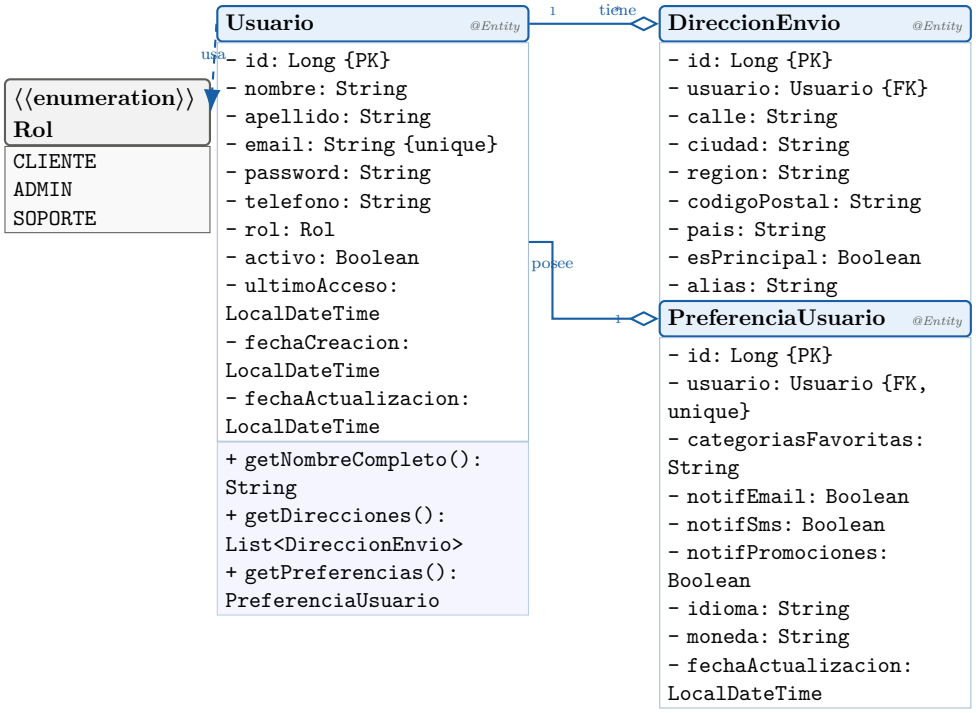

Figura 1: Diagrama UML de clases del microservicio de Usuarios (diseño lógico)

7.1 Descripción de las entidades

Usuario Entidad central del dominio. Almacena los datos de identidad del cliente, su rol en el sistema y metadatos de auditoría (`fechaCreacion`, `fechaActualizacion`, `ultimoAcceso`). La contraseña se persiste exclusivamente hasheada con BCrypt.

DireccionEnvio Entidad asociada a `Usuario` en relación uno-a-muchos. Permite gestionar múltiples direcciones de entrega por cliente, distinguiendo la dirección principal mediante el atributo `esPrincipal`.

PreferenciaUsuario Entidad en relación uno-a-uno con `Usuario`. Soporta el módulo de personalización: categorías favoritas, preferencias de notificación, idioma y moneda. Su existencia es opcional; se crea con valores por defecto al registrarse el usuario.

8 Diagrama BPMN: Ciclo de Vida de Autenticación

El diagrama BPMN modela el proceso operativo principal del microservicio: el ciclo completo de autenticación de un usuario, desde el registro inicial hasta el cierre de sesión seguro.

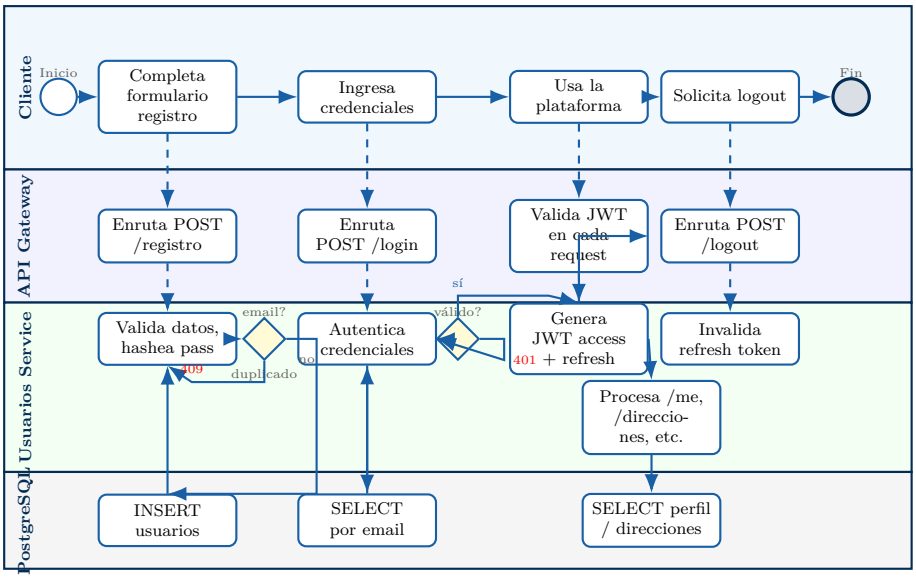

Figura 2: Diagrama BPMN del proceso de autenticación del microservicio de Usuarios

8.1 Descripción del flujo

El proceso se organiza en cuatro carriles que reflejan los actores del sistema:

1. **Cliente:** inicia el proceso completando el formulario de registro, luego introduce sus credenciales, usa la plataforma y finalmente solicita el cierre de sesión.
2. **API Gateway:** intercepta todas las solicitudes en el puerto 8080, enruta hacia el microservicio de usuarios (puerto 8081) y valida el token JWT en cada request de usuario autenticado.
3. **Usuarios Service:** ejecuta la lógica de negocio: validación y persistencia del registro, autenticación de credenciales, generación de tokens JWT y atención de operaciones de perfil.
4. **PostgreSQL:** persiste y retorna datos mediante operaciones SQL gestionadas por Spring Data JPA.

9 Documentación de la API REST

9.1 Información general

Atributo	Valor
URL base (local)	`http://localhost:8081`
URL base (gateway)	`http://localhost:8080/api/v1`
Versión API	`v1`
Formato	JSON (`Content-Type: application/json`)
Autenticación	Bearer JWT (`Authorization: Bearer <token>`)
Documentación	`http://localhost:8081/swagger-ui.html`
Health check	`http://localhost:8081/actuator/health`

Cuadro 3: Información general de la API

9.2 Módulo de Autenticación (`/api/v1/auth`)

9.2.1 POST /api/v1/auth/registro — Registrar usuario

Propiedad	Tipo	Descripción
Request Body		
`nombre`	String (req.)	Nombre del usuario
`apellido`	String (req.)	Apellido del usuario
`email`	String (req.)	Email único, formato RFC 5322
`password`	String (req.)	Mín. 8 chars, mayúscula, minúscula y número
`telefono`	String (opt.)	Formato internacional (`+56912345678`)
Respuestas		
201 Created	–	Registro exitoso; retorna `AuthResponse`
400 Bad Request	–	Validación fallida; retorna mapa de errores por campo
409 Conflict	–	Email ya registrado

```
{
  "accessToken":  "eyJhbGciOiJIUzI1...",
  "refreshToken": "eyJhbGciOiJIUzI1...",
  "tokenType":    "Bearer",
  "expiresIn":    86400000,
  "usuario": {
    "id":           1,
    "nombre":       "Maria",
    "apellido":     "Gonzalez",
    "email":        "maria.gonzalez@email.com",
    "telefono":     "+56912345678",
```

```
    "rol":            "CLIENTE",
    "activo":         true,
    "fechaCreacion":"2026-03-17T10:30:00"
  }
}
```

Listing 1: Cuerpo de respuesta 201 - AuthResponse

9.2.2 POST /api/v1/auth/login — Iniciar sesión

Propiedad	Tipo	Descripción
`email`	String (req.)	Email registrado
`password`	String (req.)	Contraseña en texto plano (se valida contra BCrypt)
200 OK	–	Autenticación exitosa; retorna `AuthResponse`
401 Unauthorized	–	Credenciales incorrectas

9.3 Módulo de Perfil (/api/v1/usuarios)

Todos los endpoints de este módulo requieren header `Authorization: Bearer <token>`.

Método	Ruta	Descripción	Respuesta
`GET`	`/api/v1/usuarios/me`	Perfil propio	200 `UsuarioResponse`
`PUT`	`/api/v1/usuarios/me`	Actualiza perfil	200 `UsuarioResponse`
`DELETE`	`/api/v1/usuarios/me`	Desactiva cuenta	204 No Content
`GET`	`/api/v1/usuarios/me/direcciones`	Lista direcciones	200 `List<DireccionResponse>`
`POST`	`/api/v1/usuarios/me/direcciones`	Agrega dirección	201 `DireccionResponse`
`DELETE`	`/api/v1/usuarios/me/direcciones/{id}`	Elimina dirección	204 No Content
`GET`	`/api/v1/usuarios/me/preferencias`	Consulta preferencias	200 `PreferenciaResponse`
`PUT`	`/api/v1/usuarios/me/preferencias`	Actualiza preferencias	200 `PreferenciaResponse`

Cuadro 4: Endpoints del módulo de perfil de usuario

9.4 Módulo Administrativo (/api/v1/admin)

Requiere rol `ADMIN`. Los endpoints estándar retornan HTTP 403 si el token no corresponde a un usuario con rol administrativo.

Método	Ruta	Descripción
GET	`/api/v1/admin/usuarios`	Lista usuarios activos (paginado)
GET	`/api/v1/admin/usuarios/buscar?q=`	Busca por nombre, apellido o email
GET	`/api/v1/admin/usuarios/{id}`	Obtiene usuario por ID
DELETE	`/api/v1/admin/usuarios/{id}`	Desactiva usuario por ID

Cuadro 5: Endpoints del módulo administrativo

9.4.1 Paginación

Los endpoints de listado soportan parámetros de paginación estándar de Spring Data:

```
GET /api/v1/admin/usuarios?page=0&size=20&sort=fechaCreacion,desc
```

Listing 2: Parámetros de paginación

9.5 Estructura de errores

Todos los errores siguen la misma estructura de respuesta para consistencia del cliente:

```
{
  "codigo":    "EMAIL_DUPLICADO",
  "mensaje":   "El email maria@email.com ya esta registrado",
  "timestamp": "2026-03-17T10:30:00",
  "detalles":  null        // Mapa campo:mensaje en errores de validacion
}
```

Listing 3: Estructura de respuesta de error

Código	HTTP	Descripción
`EMAIL_DUPLICADO`	409	Email ya registrado en el sistema
`CREDENCIALES_INVALIDAS`	401	Email o contraseña incorrectos
`USUARIO_NO_ENCONTRADO`	404	ID o email no existe en la base de datos
`DIRECCION_NO_ENCONTRADA`	404	Dirección no existe o no pertenece al usuario
`ACCESO_DENEGADO`	403	Rol insuficiente para la operación solicitada
`VALIDACION_FALLIDA`	400	Uno o más campos de entrada inválidos
`ERROR_INTERNO`	500	Error inesperado del servidor

Cuadro 6: Catálogo de códigos de error

10 Pruebas de Endpoints

Se implementaron pruebas automatizadas para validar el comportamiento de los endpoints principales del microservicio. Las pruebas se ejecutan con **JUnit 5** y **Spring MockMvc**, aislando el controlador y simulando autenticación mediante un usuario de prueba. Esto permite verificar códigos de respuesta, estructura de payloads y validaciones sin requerir servicios externos.

10.1 Alcance de las pruebas

- **UsuarioControllerTest:** pruebas de perfil, direcciones y preferencias.
- **UsuarioServiceTest:** pruebas unitarias de registro, búsqueda y desactivación.

10.2 Casos de prueba por endpoint

Método	Endpoint	Casos validados
GET	`/api/v1/usuarios/me`	Perfil propio, usuario autenticado
PUT	`/api/v1/usuarios/me`	Actualización exitosa y validación de datos inválidos
DELETE	`/api/v1/usuarios/me`	Desactivación de cuenta (204)
GET	`/api/v1/usuarios/me/direcciones`	Listado de direcciones
POST	`/api/v1/usuarios/me/direcciones`	Creación de dirección (201)
DELETE	`/api/v1/usuarios/me/direcciones/{id}`	Eliminación de dirección (204)
GET	`/api/v1/usuarios/me/preferencias`	Consulta de preferencias
PUT	`/api/v1/usuarios/me/preferencias`	Actualización de preferencias

Cuadro 7: Cobertura de pruebas en endpoints del módulo de usuario

10.3 Ejecución de pruebas

```
# Ejecutar la suite completa
mvn test

# Verificacion completa (tests + empaquetado)
mvn -DskipTest=false verify
```

Listing 4: Comandos de ejecucion de pruebas

Resultado esperado

La suite completa se ejecuta en modo aislado (MockMvc) y reporta estado *BUILD SUCCESS* cuando todos los casos validan los códigos de respuesta y la estructura de los cuerpos definidos en la API.

11 Arquitectura del Microservicio

11.1 Estructura de capas

El microservicio implementa una arquitectura de capas inspirada en *Domain-Driven Design* (DDD-lite), donde cada capa tiene una responsabilidad única y bien definida:

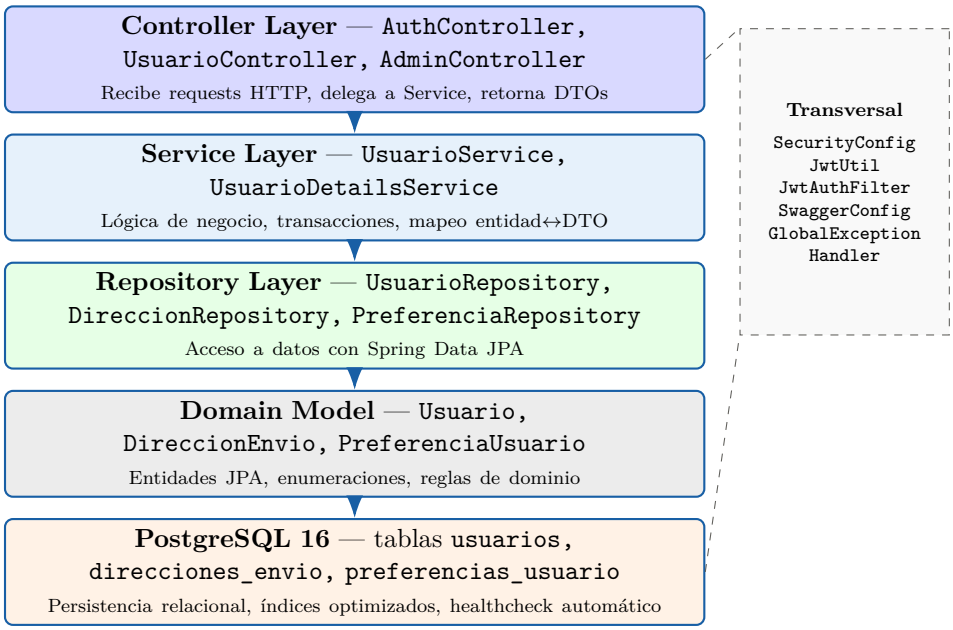

Figura 3: Arquitectura de capas del microservicio de Usuarios

11.2 Flujo de seguridad JWT

Cada request al microservicio atraviesa el filtro `JwtAuthFilter` antes de alcanzar el controlador. El flujo es el siguiente:

1. El cliente incluye el header `Authorization: Bearer <token>`.
2. `JwtAuthFilter` extrae y valida la firma del token con `JwtUtil`.
3. Si el token es válido, se carga el `UserDetails` desde `UsuarioDetailsService` y se establece el contexto de seguridad.
4. Spring Security aplica las reglas de autorización definidas en `SecurityConfig` (roles, rutas públicas, etc.).
5. Si el token es inválido o expirado, el filtro registra la advertencia y permite que Spring Security retorne HTTP 401.

12 Infraestructura y Despliegue

12.1 Contenerización con Docker

El proyecto incluye un `Dockerfile` multi-etapa que optimiza el tamaño de la imagen de producción:

```
# Etapa 1: Compilacion con Maven (imagen pesada, descartada)
FROM maven:3.9.6-eclipse-temurin-17 AS builder
RUN mvn clean package -DskipTests

# Etapa 2: Imagen de produccion (solo JRE Alpine, ~150MB)
FROM eclipse-temurin:17-jre-alpine
COPY --from=builder /app/target/*.jar app.jar
```

Listing 5: Estrategia de build multi-etapa

12.2 Docker Compose

Servicio	Imagen	Puerto	Descripción
`postgres`	`postgres:16-alpine`	5432	Base de datos principal
`usuarios-service`	build local	8081	Microservicio de usuarios
`pgadmin`	`dpage/pgadmin4`	5050	Interfaz visual (perfil `tools`)

Cuadro 8: Servicios del entorno Docker Compose

Comandos de operación:

```
# Levantar entorno completo
docker-compose up -d

# Solo base de datos (desarrollo local)
docker-compose up -d postgres

# Con pgAdmin incluido
docker-compose --profile tools up -d

# Ver logs del microservicio
docker-compose logs -f usuarios-service
```

Listing 6: Comandos Docker Compose

12.3 Variables de entorno

Variable	Default	Descripción
`DB_USER`	`postgres`	Usuario de PostgreSQL
`DB_PASSWORD`	`postgres`	Contraseña de PostgreSQL
`JWT_SECRET`	ver `application.yml`	Clave para firmar tokens JWT
`SPRING_DATASOURCE_URL`	`jdbc:postgresql://localhost:5432/shopsmart_usuarios`	URL de conexión JDBC a PostgreSQL

Cuadro 9: Variables de entorno del microservicio

13 Argumentación Arquitectónica: Monolito a Microservicio

La decisión de migrar el dominio de Usuarios está fundamentada en cinco ejes estratégicos de arquitectura:

1. **Escalabilidad selectiva:** la autenticación y gestión de perfil pueden escalar de forma independiente sin incrementar los recursos asignados a inventario o pedidos. Esto resulta crítico durante eventos de alta demanda (campañas de marketing, *Black Friday*), donde el login es el primer punto de carga.

2. **Disponibilidad aislada:** una falla en el servicio de inventario o procesamiento de pedidos no debe comprometer el acceso de los usuarios a la plataforma. El aislamiento en un proceso independiente garantiza que el dominio de identidad permanezca disponible [Richardson(2018)].

3. **Seguridad centralizada:** al concentrar toda la lógica de identidad, autenticación y control de privilegios en un único microservicio, se simplifica la auditoría, el endurecimiento de la API y la implementación de políticas de seguridad (JWT, BCrypt, RBAC).

4. **Mantenibilidad y ciclos de entrega independientes:** el equipo responsable del dominio de Usuarios puede desplegar nuevas funcionalidades (fidelización, soporte, personalización) sin coordinación con los equipos de inventario o pedidos, reduciendo el riesgo y acelerando la entrega de valor.

5. **Evolución funcional:** la arquitectura permite agregar de forma incremental capacidades avanzadas: integración con sistemas de recomendación (ML), módulo de soporte con SLA, programa de puntos de fidelización, sin afectar la estabilidad de otros dominios.

13.1 Justificación de Spring Boot sobre Django

La selección de Spring Boot frente a Django se fundamenta en los siguientes criterios técnicos aplicados al contexto ShopSmart:

Criterio	Spring Boot	Django
Concurrencia	Excelente: JVM sin GIL	Limitada: GIL de Python
Microservicios	Nativo: Spring Cloud, Resilience4j	Diseño monolítico original
Escalabilidad	Horizontal nativa en Kubernetes	Requiere stack adicional
Tipado	Estático (Java 17)	Dinámico
Ecosistema JWT	Spring Security integrado	Librería externa (DRF-JWT)
Rendimiento	Alto (JIT compilation)	Moderado

Cuadro 10: Comparación Spring Boot vs Django para el contexto ShopSmart

14 Discusión de Impacto

14.1 Beneficios esperados

La migración del dominio de Usuarios genera impacto directo en tres dimensiones:

- **Reducción del riesgo operativo:** los releases del módulo de identidad ya no requieren redesplegar el sistema completo, eliminando ventanas de mantenimiento globales.
- **Mejora en la experiencia del cliente:** tiempos de respuesta más predecibles gracias a la escalabilidad selectiva; disponibilidad del login incluso en degradación parcial del sistema.
- **Aceleración del desarrollo:** el equipo de Usuarios puede iterar de forma autónoma sobre personalización, fidelización y soporte, con menor coordinación entre equipos.

14.2 Riesgos y mitigaciones

La migración introduce retos propios de sistemas distribuidos que deben gestionarse proactivamente [Newman(2021)]:

Riesgo	Descripción	Mitigación
Latencia de red	Comunicación HTTP agrega latencia vs. llamadas in-process	API Gateway con Circuit Breaker (Resilience4j); timeouts configurados
Consistencia eventual	Datos de usuario no siempre sincronizados entre servicios	Contratos API estables; versionado de endpoints
Observabilidad	Trazabilidad más compleja en sistemas distribuidos	Spring Actuator; recomendado Zipkin/Jaeger
Seguridad token	JWT comprometido válido hasta expiración	Expiración 24h; refresh 7d; blacklist en Redis
Complejidad operacional	Más servicios implican más infraestructura	Docker Compose (dev); Kubernetes (prod)

Cuadro 11: Matriz de riesgos y mitigaciones

15 Criterios de Aceptación

Los siguientes criterios definen el umbral mínimo de aceptación del microservicio:

1. `POST /api/v1/auth/registro` retorna HTTP 201 y tokens JWT válidos con datos correctos.
2. `POST /api/v1/auth/login` retorna HTTP 200 con tokens válidos para credenciales correctas, y HTTP 401 para credenciales incorrectas.
3. `GET /api/v1/usuarios/me` retorna únicamente los datos del usuario cuyo token se presenta.
4. `PUT /api/v1/usuarios/me` actualiza perfil sin permitir escalación de privilegios (el campo `rol` es de solo lectura para el cliente).
5. Usuario sin rol ADMIN recibe HTTP 403 al intentar acceder a `/api/v1/admin/**`.
6. Los campos de perfil extendido (`telefono`, `preferencias`, `direcciones`) persisten correctamente en PostgreSQL.
7. El endpoint `/actuator/health` retorna HTTP 200 con estado `UP`.
8. La documentación Swagger está disponible en `/swagger-ui.html` con todos los endpoints documentados.

16 Conclusiones

La separación del dominio de Usuarios constituye una primera fase sólida y estratégica de modernización de la plataforma ShopSmart. La implementación con Spring Boot 3.2 demuestra cómo las herramientas del ecosistema Java permiten abordar de forma

natural los tres problemas fundamentales identificados en el monolito: escalabilidad selectiva, disponibilidad aislada y evolución funcional independiente.

El microservicio implementado cumple con los requerimientos funcionales y no funcionales especificados, incorporando mejores prácticas de seguridad (JWT + BCrypt + RBAC), observabilidad (Actuator), documentación automática (OpenAPI 3) e infraestructura reproducible (Docker multi-etapa). La arquitectura de capas garantiza mantenibilidad y facilita la incorporación de nuevos desarrolladores al equipo.

Desde una perspectiva arquitectónica, la migración está alineada con las prácticas de evolución incremental descritas por Newman [Newman(2021)] y Richardson [Richardson(2018)], reduciendo la dependencia del ciclo de vida del monolito y habilitando escalado y despliegue verdaderamente independientes.

Como pasos futuros se recomienda incorporar trazabilidad distribuida (Zipkin/Jaeger), un sistema de blacklist de tokens con Redis, y la migración progresiva de los dominios de Inventario y Pedidos para completar la transición a una arquitectura de microservicios completa.

16 Referencias

[Fowler(2015)] Fowler, M. (2015) *Microservices: a definition of this new architectural term*. Available at: https://martinfowler.com/articles/microservices.html (Accessed: 17 March 2026).

[Newman(2021)] Newman, S. (2021) *Building Microservices*. 2nd edn. Sebastopol: O'Reilly Media.

[Richardson(2018)] Richardson, C. (2018) *Microservices Patterns*. Shelter Island: Manning Publications.

[Spring(2024)] Spring Framework (2024) *Spring Boot Reference Documentation 3.2*. Available at: https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/ (Accessed: 17 March 2026).

[SpringDoc(2024)] SpringDoc OSS (2024) *SpringDoc OpenAPI 3 Library for Spring Boot*. Available at: https://springdoc.org/ (Accessed: 17 March 2026).

[OWASP(2023)] OWASP Foundation (2023) *JSON Web Token Cheat Sheet for Java*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html (Accessed: 17 March 2026).

[Docker(2024)] Docker Inc. (2024) *Docker Documentation: Multi-stage builds*. Available at: https://docs.docker.com/build/building/multi-stage/ (Accessed: 17 March 2026).